Inference-Time Repair for Robust Tool Calling Under Interface Drift

Anonymous Authors

Draft prepared for NGEN-AI 2026

Abstract. Tool-calling language models rely heavily on surface-form tool interfaces, including descriptions, parameter names, and candidate tool sets. In realistic deployments, these interfaces drift after release even when the underlying tool semantics remain unchanged. We study this problem as *interface drift* and evaluate a lightweight training-free mitigation, SchemaShield-Lite, built from canonical tool cards, validation, and one-shot repair. We instantiate interface drift using stale description examples, schema renaming, and distractor tools on DICE-BENCH, a recent benchmark for multi-round multi-party tool use. On a 50-example DICE slice with Qwen3.5-9B, accuracy drops from 0.80 to 0.72 under drift, while SchemaShield-Lite recovers performance to 0.88. On the subset of examples the base model originally solved correctly, drift reduces accuracy from 1.00 to 0.85, and SchemaShield-Lite restores it to 1.00, recovering all 6/6 drift-induced failures with zero repair harms. A second Qwen-family model shows the same clean-slice trend: SchemaShield-Lite recovers all 7/7 observed drift-induced failures with zero harms. These results suggest that inference-time repair is a practical robustness layer for tool-calling systems operating under post-deployment interface drift.

Keywords: tool calling · language models · robustness · interface drift · repair

1 Introduction

Modern language-model agents increasingly depend on external tools for retrieval, execution, planning, and actuation. In practice, however, tool use is mediated entirely through text: tool descriptions, parameter schemas, enum values, and candidate tool lists. Recent work shows that these interface details strongly affect tool selection and argument generation even when the tool semantics are unchanged [2, 1]. This creates a deployment problem: tool interfaces drift over time as developers rename parameters, revise descriptions, add examples, or expand candidate tool sets.

We study this problem as *interface drift*: benign, semantics-preserving changes to tool interfaces that occur after deployment. Our goal is not to train a new tool-calling model, but to quantify drift sensitivity and test whether a compact inference-time repair layer can recover reliability without retraining.

We focus on a practical setting motivated by limited experimentation time and short-paper scope. First, we instantiate drift with three realistic perturbations: (i) stale description examples that mention legacy field names, (ii) schema renaming, and (iii) distractor tools. Second, we evaluate recent open Qwen-family models on a 50-example slice of DICE-BENCH [3]. Third, we apply SchemaShield-Lite, a deterministic inference-time stack combining canonical tool rendering, validation, and one-shot forced-tool repair.

Our strongest result is on Qwen3.5-9B. On the 50-example DICE slice, accuracy drops from 0.80 to 0.72 under drift, while repair increases it to 0.88. More importantly, on the subset of examples the base model originally solved correctly, drift lowers accuracy from 1.00 to 0.85, and SchemaShield-Lite restores it to 1.00, recovering all 6/6 true drift-induced failures with zero harms. A second Qwen-family model, Qwen3.5-35B-A3B, shows a weaker overall baseline but the same clean-slice pattern: all 7/7 observed drift-induced failures are recovered with zero harms.

This paper makes three contributions:

1. We frame post-deployment tool-interface change as a unified robustness problem for tool-calling LLMs.
2. We introduce a compact, training-free repair pipeline suitable for short-cycle deployment settings.
3. We provide empirical evidence on DICE-BENCH that inference-time repair can substantially recover tool-calling accuracy under interface drift, especially on examples the model originally solves correctly.

2 Related Work

Recent work has established that tool interfaces are not neutral wrappers around tool use; they are part of the learning and inference problem itself. Guo et al. [2] show that rewriting tool descriptions can substantially improve agent reliability, especially when candidate tool sets are large. Faghih et al. [1] show the opposite direction: small edits to tool descriptions can dramatically shift tool preference, exposing a fragility in description-mediated tool use.

Several recent papers study robustness and recovery in tool-use systems. Zhang et al. [5] show that execution-error recovery remains a major weakness, particularly for smaller models, and propose RL-based training to improve it. Our setting is narrower and more deployment-oriented: we target inference-time robustness under interface drift rather than post-training optimization.

Benchmark work has also highlighted that realistic function calling is still far from solved. ComplexFuncBench introduces multi-step and constrained long-context function calling [6], while DICE-BENCH focuses on multi-round, multi-party dialogues with tool-related information dispersed across conversation turns [3]. We choose DICE-BENCH because its conversational structure better matches the kind of stale-doc and schema-drift failures that arise in practical tool-mediated assistants.

Compared with dataset-generation work such as APIGen [4], our objective is different. We do not create new large-scale tool-calling training data. Instead, we evaluate whether a lightweight repair layer can stabilize existing models against a specific class of post-deployment failures.

3 Problem Formulation

Let a tool be represented by an interface $I = (d, s, C)$, where d is the natural-language description, s is the argument schema, and C is the candidate tool set shown to the model. The underlying tool semantics are assumed fixed. An *interface drift* operator Δ maps I to a new interface $\Delta(I)$ that preserves tool semantics but perturbs surface form.

We study three drift types:

- **Description drift**: add stale or legacy examples to the description that refer to older field names.
- **Schema drift**: rename parameters while preserving semantics.
- **Candidate-set drift**: add distractor tools with overlapping descriptions.

Given user context x, tool interface I, and model M, the base system outputs a tool call $y = M(x, I)$. We evaluate three stages:

1. **Original**: inference on the original tool interface.
2. **Drifted**: inference on the drifted interface $\Delta(I)$.
3. **Repaired**: inference-time repair of an invalid or incomplete drifted call.

Our central question is whether an inference-time repair layer can recover the performance lost from Original to Drifted without harming examples that were already correct.

4 Method

4.1 SchemaShield-Lite

SchemaShield-Lite is a lightweight inference-time robustness layer with three parts:

Canonical tool card. Each tool is rendered into a compact canonical form containing the tool name, description, required fields, and field-level type information. This normalizes interface presentation across tools and emphasizes current field names.

Validation. After the base model emits a tool call, a validator checks tool name, required fields, and argument structure. Validation produces a structured error report rather than a binary failure signal.

One-shot repair. If the drifted call is invalid, the model receives a repair prompt containing the task context, canonical tool card, invalid call, and validation errors. Repair uses a single constrained call. In the strongest configuration we use *forced-tool repair*: the repair step is constrained to the intended tool when provider support allows it, with JSON fallback if the provider does not support named tool choice.

4.2 Repair Prompting Details

Two implementation details turned out to matter materially:

- The repair prompt explicitly instructs the model to infer all required fields from the dialogue even when the invalid call is empty.
- The repair stage uses a larger token budget than the base prediction stage. Raising repair capacity from the earlier default to 512 tokens fixed truncated repair outputs on longer DICE dialogues.

5 Experimental Setup

5.1 Benchmark and Data Slice

We evaluate on DICE-BENCH [3]. Our stage-2 evaluation uses a 50-example subset drawn from single-function round-1 dialogues, one example per diagnostic instance, exported from the official benchmark assets. This subset is large enough to provide concrete evidence while remaining practical under a short paper timeline.

5.2 Models

We evaluate two Qwen-family open models:

- **Qwen3.5-9B**
- **Qwen3.5-35B-A3B**

All experiments use hosted inference through OpenRouter routed to Venice, with temperature 0.0 for deterministic-style evaluation.

5.3 Metrics

We report:

- **Original score**: accuracy on the unmodified interface.
- **Drifted score**: accuracy under interface drift.
- **Repaired score**: accuracy after one-shot repair.
- **Originally-correct slice**: the subset of examples that the model solves correctly before drift.
- **Repair recoveries**: number of drift-induced failures recovered by repair.
- **Repair harms**: number of drifted-correct examples broken by repair.

The originally-correct slice is critical. A model with a weak baseline can look stronger under drift purely because of stochastic variation or baseline quality limitations. The clean slice isolates *true drift harm*.

Table 1. Main results on the 50-example DICE stage-2 slice.

Model	Orig.	Drift.	Repair	Orig.-correct n	Drift on clean	Repair on clean
Qwen3.5-9B	0.80	0.72	0.88	40	0.85	1.00
Qwen3.5-35B-A3B	0.66	0.68	0.86	33	0.7879	1.00

Table 2. Repair behavior on the originally-correct slice.

Model	Drift misses	Recoveries	Harms
Qwen3.5-9B	6	6	0
Qwen3.5-35B-A3B	7	7	0

6 Results

6.1 Main Results

Table 1 shows the main 50-example results. The clearest global result is on Qwen3.5-9B: drift lowers accuracy from 0.80 to 0.72, and repair raises it to 0.88. The 35B model has a weaker clean baseline on this slice, so its global score does not cleanly degrade under drift. However, both models show the same behavior on the originally-correct slice: drift causes real failures and repair recovers all observed drift-induced misses, with zero harms.

Table 2 focuses on repair behavior for true drift misses. For Qwen3.5-9B, repair recovers all 6/6 drift-induced failures. For Qwen3.5-35B-A3B, repair recovers all 7/7 observed failures. In both cases, repair introduces zero harms on the originally-correct slice.

6.2 Ablation on Repair Strength

Table 3 summarizes the progression of the 9B model as the repair pipeline was strengthened. Moving from the 20-example pilot to the 50-example slice already retained the same qualitative trend. Increasing repair capacity to 512 tokens removed the final unrecovered drift miss from the originally-correct slice and improved repaired accuracy from 0.86 to 0.88.

6.3 Error Analysis

The remaining global errors in the best 9B run are not drift-repair failures. They are mostly baseline semantic mismatches that cap the original score, such as shorter or underspecified content fields in reminder-style tasks. This distinction matters: the method is designed to recover *drift-induced interface failures*, not to solve all semantic-generation errors in the benchmark.

The most important implementation lesson came from a failure case in appointment scheduling. Repair initially failed not because the model could not

Table 3. Ablation on the Qwen3.5-9B repair pipeline.

Setting	n	Orig.	Drift.	Repair
Pilot after forced-tool repair	20	0.75	0.70	0.85
Stage 2 before higher repair budget	50	0.82	0.74	0.86
Stage 2 with 512 repair tokens	50	0.80	0.72	0.88

infer the fields, but because the repair output was truncated by the token budget. After increasing the repair budget, the same case repaired cleanly. This suggests that robustness under drift is partly a prompting and systems problem, not only a modeling problem.

7 Discussion

Our results support a narrow but practical conclusion: inference-time repair is a viable robustness layer for tool-calling systems operating under interface drift. The gain is especially clear when evaluation is restricted to examples the base model originally solved correctly. This slice prevents weak baseline behavior from masking or exaggerating drift effects.

At the same time, the comparison across the two Qwen-family models highlights a reporting caveat. Global scores alone can be misleading: the 35B model has a weaker baseline on this particular slice, which makes its global drift pattern less clean. Yet on the originally-correct slice, the repair behavior is just as strong. For short-paper reporting, we therefore recommend pairing global accuracy with a clean slice defined by originally-correct examples.

8 Limitations

This paper has several limitations. First, the main benchmark evidence comes from a single benchmark family and a 50-example stage-2 slice rather than a full large-scale benchmark sweep. Second, our drift operators focus on stale descriptions, renamed fields, and distractors; other forms of interface drift remain untested. Third, the evaluation uses hosted inference rather than self-hosted deployments, which may introduce provider-specific behavior. Finally, while the results are strong on the originally-correct slice, they do not imply that the method fixes all semantic generation errors or improves weak baseline tool-calling models in general.

9 Conclusion

We presented SchemaShield-Lite, a lightweight inference-time approach for improving robust tool calling under interface drift. On a 50-example DICE-BENCH slice, our strongest configuration improves Qwen3.5-9B from 0.72 to 0.88 after

drift, and recovers all 6/6 true drift-induced failures on originally-correct examples with zero harms. A second Qwen-family model shows the same clean-slice pattern, recovering all 7/7 observed drift-induced failures with zero harms. These results suggest that inference-time repair is a practical and deployable complement to model training for tool-mediated language systems.

References

1. Faghih, K., Wang, W., Cheng, Y., Bharti, S., Sriramanan, G., Balasubramanian, S., Hosseini, P., Feizi, S.: Gaming tool preferences in agentic llms. arXiv preprint arXiv:2505.18135 (2025)
2. Guo, Y., et al.: Learning to rewrite tool descriptions for reliable llm-agent tool use. arXiv preprint arXiv:2602.20426 (2026)
3. Jang, K., Lee, D., Kim, K., Heo, D., Lee, T., Kim, W., Suh, B.: Dice-bench: Evaluating the tool-use capabilities of large language models in multi-round, multi-party dialogues. arXiv preprint arXiv:2506.22853 (2025)
4. Liu, Z., Hoang, T., Zhang, J., Zhu, M., Lan, T., Kokane, S., Tan, J., Yao, W., Liu, Z., Feng, Y., Murthy, R., Yang, L., Savarese, S., Niebles, J.C., Wang, H., Heinecke, S., Xiong, C.: Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets. arXiv preprint arXiv:2406.18518 (2024)
5. Zhang, Y., et al.: Robust tool use via fission-grpo. arXiv preprint arXiv:2601.15625 (2026)
6. Zhong, Y., et al.: Complexfuncbench: Exploring multi-function calling in large language models. arXiv preprint arXiv:2501.10132 (2025)